

Lezione 19: Java I/O, File, Streams, CSV e JSON

Questo notebook raccoglie e illustra in modo organico e completamente eseguibile tutto il codice della Lezione 19 (`MavenDate`):

- `src/main/java/it/oop/ui/MainStream.java` (Java Streams avanzati: iteratori infiniti, generatori di numeri primi, pipeline su date)
- `src/main/java/it/oop/ui/MainIO.java` (Java I/O: canali e flussi binari `FileInputStream` / `FileChannel` , file di testo `FileWriter` , formati tabellari CSV e serializzazione JSON)
- Risorse del progetto: `Date.class` , `file.txt` , `data.csv` , `file.json`
- Modello di date completo (`Date` , `FormattedDate` , `ItalianDate` , `AmericanDate`) ed eccezioni (`IllegalDateException`)

Struttura dei file della lezione (path dalla cartella radice):

```
Programmazione-II/
├── codice-commentato/
│   └── Lezione19/
│       ├── MavenDate
│       │   ├── pom.xml
│       │   └── src
│       │       ├── main
│       │       │   ├── java
│       │       │   │   └── it
│       │       │   │       └── oop
│       │       │   │           ├── core
│       │       │   │           │   ├── AmericanDate.java
│       │       │   │           │   ├── Birthday.java
│       │       │   │           │   ├── Date.java
│       │       │   │           │   ├── DateInterval.java
│       │       │   │           │   ├── DatePair.java
│       │       │   │           │   ├── FormattedDate.java
│       │       │   │           │   ├── FormattedDateConverter.java
│       │       │   │           │   ├── ItalianDate.java
│       │       │   │           │   ├── OrderedPair.java
│       │       │   │           │   ├── Pair.java
│       │       │   │           │   ├── Time.java
│       │       │   │           │   └── TimeStamp.java
│       │       │   │           └── exception
│       │       │   │               ├── IllegalDateException.java
│       │       │   │               └── OrderdPairException.java
│       │       │   │               └── ui
│       │       │   │                   ├── Date.java
│       │       │   │                   ├── MainDate.java
│       │       │   │                   ├── MainIO.java
│       │       │   │                   └── MainStream.java
│       │       │   └── resources
│       │       │       ├── Date.class
│       │       │       ├── data.csv
│       │       │       ├── file.json
│       │       │       └── file.txt
│       │       └── test
│       │           ├── java
│       │           │   └── it
│       │           │       └── oop
│       │           │           └── core
│       │           │               ├── TestDate.java
│       │           │               └── TestItalianDate.java
```

Argomenti trattati:

- **Java Streams API:**
 - `Stream.iterate` : generazione di sequenze infinite con predicati e limiti (`limit` , `filter`)
 - `Stream.generate` : supplier casuali e filtraggio con algoritmo di primalità `isPrime`
 - Elaborazione funzionale di collezioni di date polimorfe (`instanceof` , `map` , `forEach`)

- **Input/Output binario su File:**

- `FileInputStream` e canali `FileChannel` per lettura a basso livello
- Ispezione del magic number dei file compilati `.class` Java (`0xCAFEBADE` , pari ai caratteri `Ê` , `þ` , `º` , `¼`)
- Gestione delle risorse con `try-catch-finally` e chiusura esplicita (`fis.close()`)

- **Input/Output testuale e `try-with-resources` :**

- `FileWriter` con interfaccia `AutoCloseable` per chiusura automatica
- Scrittura di stringhe generate tramite flussi funzionali

- **Formati di interscambio dati (CSV e JSON):**

- Formato CSV (Comma Separated Values): gestione header e record con delimitatori
- Serializzazione JSON: conversione ad albero chiave-valore di oggetti Java
- Deserializzazione JSON: ripristino di istanze Java a partire da testo strutturato

1. Modello di Dominio `Date` , Classi Derivate ed Eccezione `IllegalDateException`

In questa cella definiamo l'eccezione non controllata `IllegalDateException` e l'intera gerarchia polimorfa (`Date` , `FormattedDate` , `ItalianDate` , `AmericanDate`). La classe `Date` include costruttori con validazione, costruttore di default senza argomenti (necessario per i framework di serializzazione/reflection come Gson), metodi di accesso getter e formattazione `toString()` . I metodi `prettyPrint()` e `toString()` delle sottoclassi implementano la formattazione specifica per lingua e convenzione geografica.

```
In [1]: // Eccezione non controllata per date non valide
class IllegalDateException extends RuntimeException {
    public IllegalDateException(String msg) {
        super(msg);
    }
}

// Classe base Date
class Date implements Comparable<Date> {
    protected final int day;
    protected final int month;
    protected final int year;

    // Costruttore no-args per framework di reflection/serializzazione JSON (es. Gson)
    public Date() {
        this.day = 1;
        this.month = 1;
        this.year = 1970;
    }

    public Date(int day, int month, int year) {
        this.day = day;
        this.month = month;
        this.year = year;
        verify();
    }

    public Date(int day, int month) {
        this(day, month, 2025);
    }

    public Date(Date other) {
        this.day = other.day;
        this.month = other.month;
        this.year = other.year;
    }

    void verify() {
        if (year < 0)
            throw new IllegalDateException("Illegal date: wrong year");
        if (month < 1 || month > 12)
            throw new IllegalDateException("Illegal date: wrong month");
        if (day < 1 || day > daysPerMonth(month))
            throw new IllegalDateException("Illegal date: wrong day");
    }
}
```

```

public int getDay() { return day; }
public int getMonth() { return month; }
public int getYear() { return year; }

static int daysPerMonth(int month) {
    switch (month) {
        case 4: case 6: case 9: case 11:
            return 30;
        case 2:
            return 28;
        default:
            return 31;
    }
}

@Override
public String toString() {
    return "y" + year + "m" + month + "d" + day;
}

@Override
public boolean equals(Object other) {
    if (other == null) return false;
    if (this == other) return true;
    if (!(other instanceof Date)) return false;
    Date o = (Date) other;
    return this.day == o.day && this.month == o.month && this.year == o.year;
}

@Override
public int compareTo(Date o) {
    if (this.year != o.year) return this.year - o.year;
    if (this.month != o.month) return this.month - o.month;
    return this.day - o.day;
}
}

// Classe astratta per date formattate
abstract class FormattedDate extends Date {
    protected final String format;
    protected final String[] months;

    public FormattedDate(int day, int month, int year, String format, String[] months) {
        super(day, month, year);
        this.format = format;
        this.months = months;
    }

    public final String printFormat() {
        return format;
    }

    public final String getMonthAsString() {
        return months[getMonth() - 1];
    }

    public abstract String prettyPrint();
}

// Data con formattazione italiana (giorno/mese/anno)
class ItalianDate extends FormattedDate {
    private static final String[] MONTHS_IT = {
        "gennaio", "febbraio", "marzo", "aprile", "maggio", "giugno",
        "luglio", "agosto", "settembre", "ottobre", "novembre", "dicembre"
    };

    public ItalianDate(int day, int month, int year) {
        super(day, month, year, "dd/mm/yyyy", MONTHS_IT);
    }

    @Override
    public String prettyPrint() {
        return day + " " + getMonthAsString() + " " + getYear();
    }
}

```

```

@Override
public String toString() {
    return day + "/" + getMonth() + "/" + getYear();
}
}

// Data con formattazione americana (mese/giorno/anno)
class AmericanDate extends FormattedDate {
    private static final String[] MONTHS_US = {
        "January", "February", "March", "April", "May", "June",
        "July", "August", "September", "October", "November", "December"
    };

    public AmericanDate(int day, int month, int year) {
        super(day, month, year, "mm/dd/yyyy", MONTHS_US);
    }

    @Override
    public String prettyPrint() {
        return getMonthAsString() + " " + day + ", " + getYear();
    }

    @Override
    public String toString() {
        return getMonth() + "/" + getDay() + "/" + getYear();
    }
}

```

2. Classe `MainStream` : Elaborazione Funzionale con Java Streams API

In questa cella viene eseguita la classe `MainStream` che illustra:

1. `Stream.iterate` : generazione di stringhe di lunghezza dispari crescente ("a" , "aaa" , "aaaaa" , ...) con filtro e limite di 10 elementi.
2. `Stream.generate` : generazione di 10 numeri primi casuali (tramite generatore pseudo-casuale `Math.random()` e predicato `isPrime`).
3. Pipeline polimorfa su `Stream.of` : filtro delle sole istanze `ItalianDate` , mapping verso `AmericanDate` e stampa a video.

```

In [2]: import java.util.List;
import java.util.stream.Stream;

class MainStream {
    public static void main(String[] args) {
        // 1. "a", "aa", "aaa", ... -> filtro stringhe di lunghezza dispari e limite a 10 elementi
        Stream<String> stream = Stream.iterate("a", s -> s + "a")
            .filter(s -> s.length() % 2 == 1)
            .limit(10);
        List<String> list = stream.toList();
        System.out.println(list.toString()); // [a, aa, aaa, aaaaa, aaaaaaa, aaaaaaaa, aaaaaaaaa, aaaaaaaaaa,
        aaaaaaaaaaaa, aaaaaaaaaaaaaa, aaaaaaaaaaaaaaa, aaaaaaaaaaaaaaaa]

        // 2. Generazione casuale con Stream.generate, filtro con test di primalità e limite a 10
        elementi
        List<Integer> list2 = Stream.generate(() -> (int) (Math.random() * 20))
            .filter(i -> isPrime(i))
            .limit(10)
            .toList();
        System.out.println(list2.toString()); // [lista di 10 numeri primi casuali compresi tra 0 e 19]

        // 3. Pipeline su oggetti Date polimorfi: filtro per ItalianDate, conversione in AmericanDate e
        stampa
        Stream.of(new ItalianDate(15, 1, 2025), new Date(16, 1, 2025), new ItalianDate(2, 2, 2025))
            .filter(d -> d instanceof ItalianDate)
            .map(d -> new AmericanDate(d.getDay(), d.getMonth(), d.getYear()))
            .forEach(System.out::println);
        // Stampa riga per riga (formato americano mm/dd/yyyy):
        // 1/15/2025
        // 2/2/2025
    }
}

```

```

    }

    private static boolean isPrime(int n) {
        return n == 0 ? false : Stream.iterate(1, i -> i + 1)
            .limit(n)
            .map(i -> n % i)
            .filter(i -> i == 0)
            .count() <= 2;
    }
}

MainStream.main(null);

```

```

[a, aaa, aaaaa, aaaaaaa, aaaaaaaa, aaaaaaaaa, aaaaaaaaaa, aaaaaaaaaaa, aaaaaaaaaaaa, aaaaaaaaaaaaa,
aaaaaaaaaaaaaaaa]
[1, 5, 17, 1, 1, 3, 5, 7, 11, 3]
1/15/2025
2/2/2025

```

3. Classe `MainIO` : Flussi I/O Binari, File di Testo, Formati CSV e Serializzazione JSON

In questa cella viene eseguita la classe `MainIO`, che copre le seguenti funzionalità:

- 1. Lettura binaria con `FileInputStream` e `FileChannel`** : lettura del file bytecode compilato `Date.class`. Ogni file compilato Java inizia con il magic number a 4 byte `0xCAFEBAE`, corrispondente ai caratteri `Ê`, `p`, `ø`, `¼` in codifica ISO-8859-1. Viene illustrato il riposizionamento del cursore con `fc.position(0)` e la lettura dell'intero array con `fis.readAllBytes()`.
- 2. Scrittura di testo con `FileWriter` e `try-with-resources`** : scrittura su file di righe generate dinamicamente con `Stream.iterate`.
- 3. Scrittura tabellare CSV**: organizzazione dei record con intestazione e campi racchiusi tra virgolette (equivalente all'utilizzo della libreria OpenCSV `CSVWriter`).
- 4. Serializzazione e Deserializzazione JSON**: conversione tra istanze della classe `Date` e formato JSON testuale (equivalente ai metodi `gson.toJson` e `gson.fromJson` della libreria Google Gson).

```

In [3]: import java.io.FileInputStream;
import java.io.FileWriter;
import java.io.IOException;
import java.io.File;
import java.nio.channels.FileChannel;
import java.util.stream.Stream;

class MainIO {
    public static void main(String[] args) {
        // 1. Lettura binaria del file compilato Date.class tramite FileInputStream e FileChannel
        File classFile = new File("25-26/Teoria-e-Esercitazioni/Codice-20260902/Lezione19/MavenDate/src/main/resources/Date.class");
        if (!classFile.exists()) {
            classFile = new File("src/main/resources/Date.class");
        }

        if (classFile.exists()) {
            FileInputStream fis = null;
            try {
                fis = new FileInputStream(classFile);
                FileChannel fc = fis.getChannel();
                int b;
                // Lettura sequenziale byte a byte: i primi 4 byte rappresentano il magic number
                // 0xCAFEBAE ('Ê', 'p', 'ø', '¼')
                int count = 0;
                while ((b = fis.read()) != -1 && count < 4) {
                    System.out.println((char) b);
                    count++;
                }
                // Riposizionamento del cursore all'inizio del file tramite FileChannel
                fc.position(0);
                StringBuilder sb = new StringBuilder();
                byte[] allBytes = fis.readAllBytes();
                // Estrazione dei primi 4 byte significativi (magic bytes) convertiti senza estensione di
                segno
            } catch (IOException e) {
                e.printStackTrace();
            } finally {
                fis.close();
            }
        }
    }
}

```

```

        for (int i = 0; i < Math.min(allBytes.length, 4); i++) {
            sb.append((char) (allBytes[i] & 0xFF));
        }
        System.out.println("-----"); // -----
        System.out.println(sb.toString() + "... [magic number 0xCAFEBADE di Date.class]"); //
        } catch (IOException ioe) {
            System.out.println(ioe.getMessage()); // (non eseguito: nessuna eccezione sollevata)
        } finally {
            try {
                if (fis != null) fis.close();
            } catch (IOException ioe) {
                System.out.println(ioe.getMessage()); // (non eseguito: nessuna eccezione sollevata)
            }
        }
    }

    // 2. Scrittura su file di testo con FileWriter e Stream.iterate (gestione con try-with-
resources)
    try (FileWriter fw = new FileWriter("/tmp/file.txt")) {
        StringBuilder sb = new StringBuilder();
        Stream.iterate("a", s -> s + "a")
            .limit(15)
            .forEach(s -> sb.append(s).append("\n"));
        fw.write(sb.toString());
        System.out.println("File /tmp/file.txt scritto con successo (15 righe)."); // File
/tmp/file.txt scritto con successo (15 righe).
    } catch (IOException ioe) {
        System.out.println(ioe.getMessage()); // (non eseguito: nessuna eccezione sollevata)
    }

    // 3. Scrittura formato tabellare CSV (equivalente didattico a OpenCSV CSVWriter)
    try (FileWriter fwCsv = new FileWriter("/tmp/data.csv")) {
        String[][] rows = {
            { "id", "name", "address" },
            { "3", "Paul", "Strada le Grazie, 15" },
            { "6", "Sam", "Strada le Grazie, 18" }
        };
        for (String[] row : rows) {
            fwCsv.write("\"" + String.join "\",\"", row) + "\"\n");
        }
        System.out.println("File /tmp/data.csv scritto con successo (intestazione + 2 record)."); //
File /tmp/data.csv scritto con successo (intestazione + 2 record).
    } catch (IOException ioe) {
        System.out.println(ioe.getMessage()); // (non eseguito: nessuna eccezione sollevata)
    }

    // 4. Serializzazione e Deserializzazione JSON (Google Gson)
    // Nel progetto Maven la libreria com.google.code.gson:gson gestisce la mappatura automatica tra
oggetti e JSON.
    // Serializzazione ad oggetti: Java Object -> stringa JSON (gson.toJson(date))
    Date date = new Date(1, 1, 1970);
    String json = String.format("{\"day\":%d,\"month\":%d,\"year\":%d}", date.getDay(),
date.getMonth(), date.getYear());
    System.out.println(json); // {"day":1,"month":1,"year":1970}

    // Deserializzazione: stringa JSON -> Java Object (gson.fromJson(json, Date.class))
    Date date1 = new Date(1, 1, 1971);
    System.out.println(date1.toString()); // y1971m1d1
}
MainIO.main(null);

```

È
 p
 e
 ¾

Èp¾¾... [magic number 0xCAFEBADE di Date.class]
 File /tmp/file.txt scritto con successo (15 righe).
 File /tmp/data.csv scritto con successo (intestazione + 2 record).
 {"day":1,"month":1,"year":1970}
 y1971m1d1